

# Compositional Taylor expansion in additive and non-additive models of Linear Logic

Aymeric Walch

Université Paris Cité, CNRS, IRIF

October 9th, 2025

# This thesis is about

# **denotational semantics**

---

```
def pow_two(n : int) -> int :  
  res = 1  
  for i in range(n)  
    res = 2 * res  
  return res
```

---

Reading code is tiring...

---

```
def pow_two(n : int) -> int :  
  # Compute 2 to the power n  
  res = 1  
  for i in range(n)  
    res = 2 * res  
  return res
```

---

Guessing is tiring...

Specification is better!

---

```
def pow_two(n : int) -> int :  
  # Compute 2 to the power n
```



---

Guessing is tiring...

Specification is better!

Programming is **modular**

# From specification to semantics

Specification is based on **words**. Not always enough.

- ▶ Critical software: **Plane pilot**
- ▶ Design of programming languages: **Ocaml, Haskell, Rust**
- ▶ Mix programming languages with maths: **automated differentiation, probabilistic programming.**

# From specification to semantics

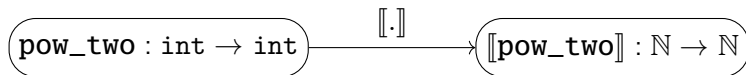
Specification is based on **words**. Not always enough.

- ▶ Critical software: **Plane pilot**
- ▶ Design of programming languages: **Ocaml, Haskell, Rust**
- ▶ Mix programming languages with maths: **automated differentiation, probabilistic programming.**

**Semantics:** specification based on **maths**

# Denotational semantics

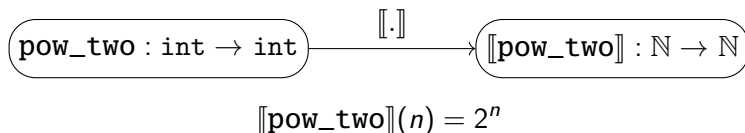
**Denotational semantics:** interpret programs as functions.





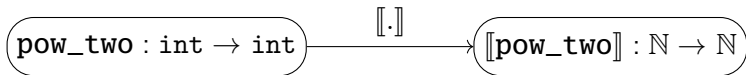
# Denotational semantics

**Denotational semantics:** interpret programs as functions.

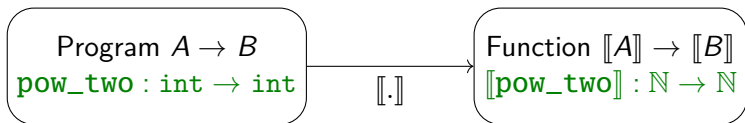
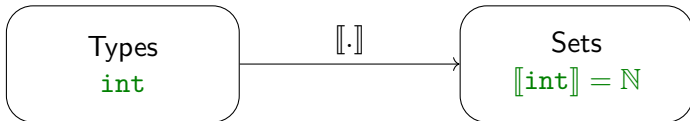


# Denotational semantics

**Denotational semantics:** interpret programs as functions.



$$\llbracket \text{pow\_two} \rrbracket(n) = 2^n$$



# Denotational semantics is compositional

Programming is modular.

---

```
def pow_four(n : int) -> n :  
    # Compute 4 to the power n  
    return (pow_two(n) * pow_two(n))
```

---

# Denotational semantics is compositional

Programming is modular.

---

```
def pow_four(n : int) -> n :  
    # Compute 4 to the power n  
    return (pow_two(n) * pow_two(n))
```

---

So is semantics

$$\llbracket \text{pow\_four} \rrbracket(n) = \llbracket \text{pow\_two} \rrbracket(n) \times \llbracket \text{pow\_two} \rrbracket(n)$$

# Denotational semantics is compositional

Programming is modular.

---

```
def pow_four(n : int) -> n :  
    # Compute 4 to the power n  
    return (pow_two(n) * pow_two(n))
```

---

So is semantics

$$\llbracket \text{pow\_four} \rrbracket(n) = \llbracket \text{pow\_two} \rrbracket(n) \times \llbracket \text{pow\_two} \rrbracket(n)$$

## Compositionality

$$\llbracket \text{pow\_four} \rrbracket = f \circ \llbracket \text{pow\_two} \rrbracket$$

with  $f(m) = m \times m$

We need a mathematical theory  
of composition

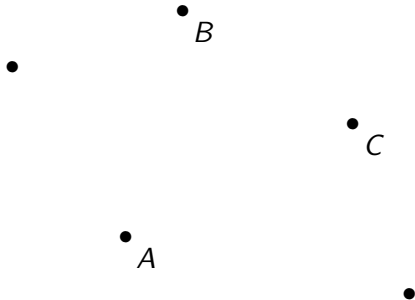
# Category theory!!!!

A category is

# Category theory!!!!

A category is

- Objects  $A, B, C \dots$

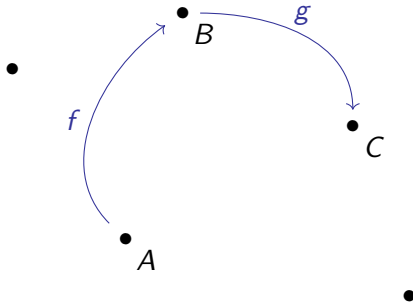




# Category theory!!!!

A category is

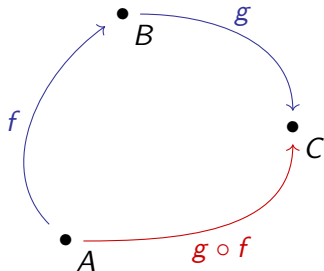
- ▶ Objects  $A, B, C \dots$
- ▶ Morphisms  $f, g, h \dots$



# Category theory!!!!

A category is

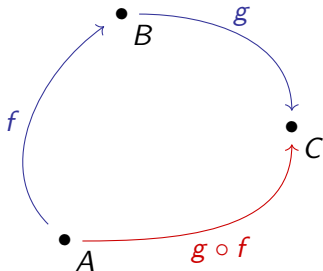
- ▶ Objects  $A, B, C \dots$
- ▶ Morphisms  $f, g, h \dots$
- ▶ Composition  $\_ \circ \_$



# Category theory!!!!

A category is

- ▶ Objects  $A, B, C \dots$  Sets
- ▶ Morphisms  $f, g, h \dots$  Functions
- ▶ Composition  $\_ \circ \_$  Function composition



**Syntax****Semantics in  
category of sets and  
functions****Generalization to  
other categories**

Types  
`int`

Sets  
 $\mathbb{N}$

???

Program  $A \rightarrow B$   
`pow_two`  
`int  $\rightarrow$  int`

Function  $A \rightarrow B$   
 $\llbracket \text{pow\_two} \rrbracket$   
 $\mathbb{N} \rightarrow \mathbb{N}$

???

## Syntax

Types  
`int`

Program  $A \rightarrow B$   
`pow_two`  
`int → int`

## Semantics in category of sets and functions

Sets  
 $\mathbb{N}$

Function  $A \rightarrow B$   
 $\llbracket \text{pow\_two} \rrbracket$   
 $\mathbb{N} \rightarrow \mathbb{N}$

## Generalization to other categories

Objects  
Object of integers  $\mathbb{N}$

???

## Syntax

Types  
`int`

Program  $A \rightarrow B$   
`pow_two`  
`int → int`

## Semantics in category of sets and functions

Sets  
 $\mathbb{N}$

Function  $A \rightarrow B$   
 $\llbracket \text{pow\_two} \rrbracket$   
 $\mathbb{N} \rightarrow \mathbb{N}$

## Generalization to other categories

Objects  
Object of integers  $\mathbb{N}$

Morphism  $A \rightarrow B$   
 $\llbracket \text{pow\_two} \rrbracket$   
 $\mathbb{N} \rightarrow \mathbb{N}$

**Syntax****Semantics in  
category of sets and  
functions****Generalization to  
other categories**Types  
`int`Sets  
 $\mathbb{N}$ Objects  
Object of integers  $\mathbb{N}$ Program  $A \rightarrow B$   
`pow_two`  
`int  $\rightarrow$  int`Function  $A \rightarrow B$   
`[[pow_two]]`  
 $\mathbb{N} \rightarrow \mathbb{N}$ Morphism  $A \rightarrow B$   
`[[pow_two]]`  
 $\mathbb{N} \rightarrow \mathbb{N}$ 

What about type constructors ?

Product type  
`int  $\times$  int`Cartesian product  
 $\mathbb{N} \times \mathbb{N}$ 

???

**Syntax****Semantics in  
category of sets and  
functions****Generalization to  
other categories**

Types  
 $\text{int}$

Sets  
 $\mathbb{N}$

Objects  
Object of integers  $\mathbb{N}$

Program  $A \rightarrow B$   
 $\text{pow\_two}$   
 $\text{int} \rightarrow \text{int}$

Function  $A \rightarrow B$   
 $\llbracket \text{pow\_two} \rrbracket$   
 $\mathbb{N} \rightarrow \mathbb{N}$

Morphism  $A \rightarrow B$   
 $\llbracket \text{pow\_two} \rrbracket$   
 $\mathbb{N} \rightarrow \mathbb{N}$

What about type constructors ?

Product type  
 $\text{int} \times \text{int}$

Cartesian product  
 $\mathbb{N} \times \mathbb{N}$

**Categorical product**  
 $\mathbb{N} \times \mathbb{N}$



**Syntax****Semantics in  
category of sets and  
functions****Generalization to  
other categories**Types  
 $\text{int}$ Sets  
 $\mathbb{N}$ Objects  
Object of integers  $\mathbb{N}$ Program  $A \rightarrow B$   
 $\text{pow\_two}$   
 $\text{int} \rightarrow \text{int}$ Function  $A \rightarrow B$   
 $\llbracket \text{pow\_two} \rrbracket$   
 $\mathbb{N} \rightarrow \mathbb{N}$ Morphism  $A \rightarrow B$   
 $\llbracket \text{pow\_two} \rrbracket$   
 $\mathbb{N} \rightarrow \mathbb{N}$ 

What about type constructors ?

Product type  
 $\text{int} \times \text{int}$ Cartesian product  
 $\mathbb{N} \times \mathbb{N}$ **Categorical product**  
 $\mathbb{N} \times \mathbb{N}$ Function type  
 $\text{int} \rightarrow \text{int}$ Set of functions  
 $\mathbb{N} \rightarrow \mathbb{N}$ 

???

**Syntax****Semantics in  
category of sets and  
functions****Generalization to  
other categories**Types  
 $\text{int}$ Sets  
 $\mathbb{N}$ Objects  
Object of integers  $\mathbb{N}$ Program  $A \rightarrow B$   
 $\text{pow\_two}$   
 $\text{int} \rightarrow \text{int}$ Function  $A \rightarrow B$   
 $\llbracket \text{pow\_two} \rrbracket$   
 $\mathbb{N} \rightarrow \mathbb{N}$ Morphism  $A \rightarrow B$   
 $\llbracket \text{pow\_two} \rrbracket$   
 $\mathbb{N} \rightarrow \mathbb{N}$ 

What about type constructors ?

Product type  
 $\text{int} \times \text{int}$ Cartesian product  
 $\mathbb{N} \times \mathbb{N}$ **Categorical product**  
 $\mathbb{N} \times \mathbb{N}$ Function type  
 $\text{int} \rightarrow \text{int}$ Set of functions  
 $\mathbb{N} \rightarrow \mathbb{N}$ **Internal hom**  
 $\mathbb{N} \multimap \mathbb{N}$

**Syntax****Semantics in  
category of sets and  
functions****Generalization to  
other categories**

Types

`int`

Sets

 $\mathbb{N}$ 

Objects

Object of integers  $\mathbb{N}$ Program  $A \rightarrow B$ `pow_two`  
 $\text{int} \rightarrow \text{int}$ Function  $A \rightarrow B$  $\llbracket \text{pow\_two} \rrbracket$   
 $\mathbb{N} \rightarrow \mathbb{N}$ Morphism  $A \rightarrow B$  $\llbracket \text{pow\_two} \rrbracket$   
 $\mathbb{N} \rightarrow \mathbb{N}$ 

Cartesian closed category

Category + Product + Internal Hom

Product type

 $\text{int} \times \text{int}$ 

Cartesian product

 $\mathbb{N} \times \mathbb{N}$ **Categorical product** $\mathbb{N} \times \mathbb{N}$ 

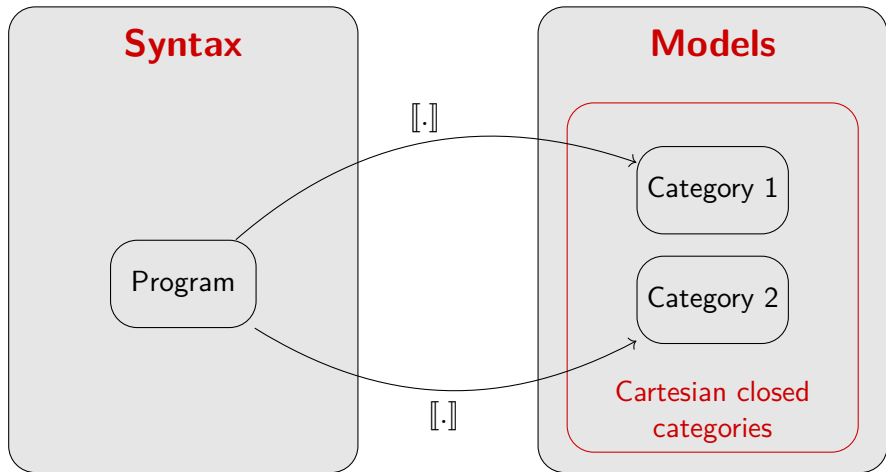
Function type

 $\text{int} \rightarrow \text{int}$ 

Set of functions

 $\mathbb{N} \rightarrow \mathbb{N}$ **Internal hom** $\mathbb{N} \multimap \mathbb{N}$

# A recipe for models



# Compositional Taylor expansion in additive and non-additive **models of Linear Logic**

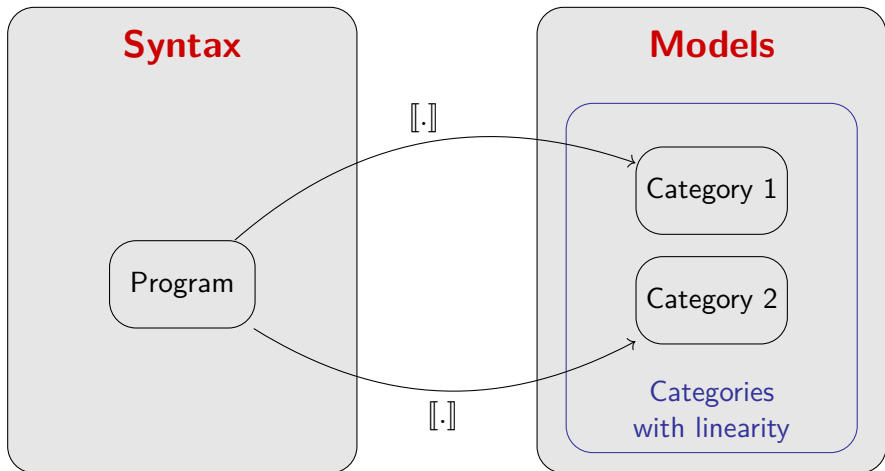
# Linear Logic

Girard 1987 observed in some cartesian closed categories a **decomposition**

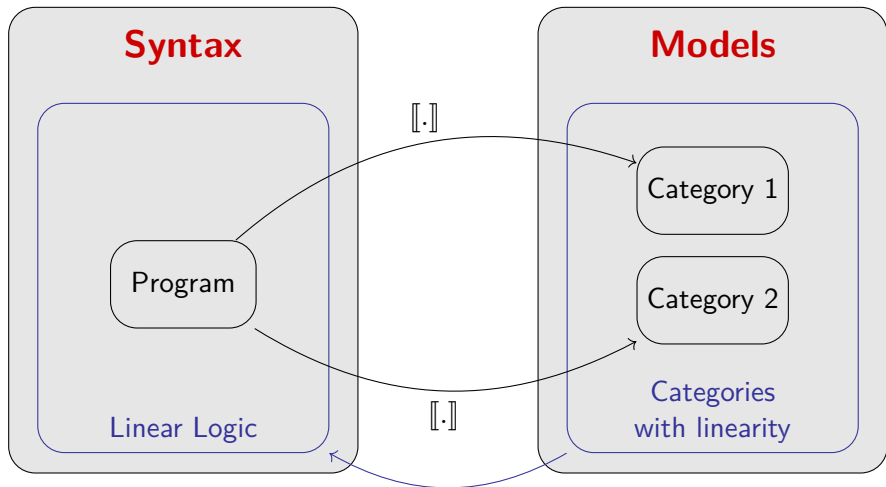
$$A \multimap B = !A \multimap B$$

- ▶  $\_ \multimap \_$  : object that describes **linear** morphisms;
- ▶  $!\_$  is a repetition operator.

# Reverse denotational semantics

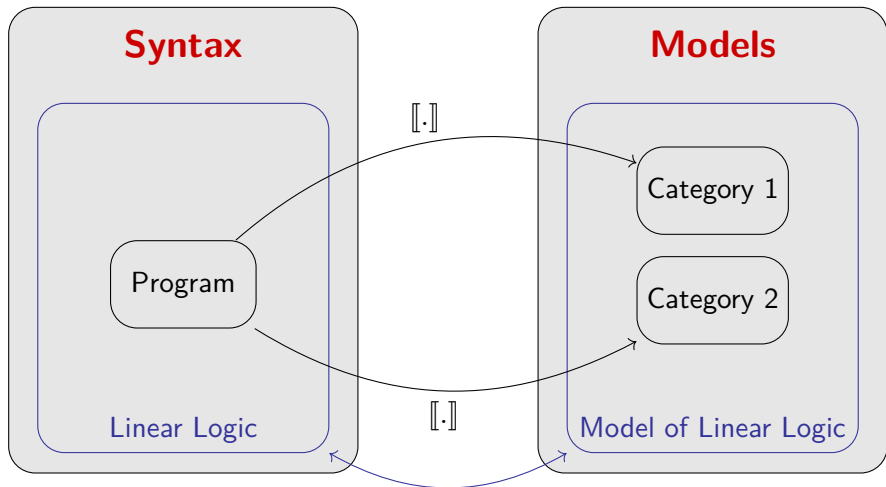


# Reverse denotational semantics





# Reverse denotational semantics



# Linear Logic: between syntax and semantics

## Syntax

- ▶ Linear programs  $A \multimap B$   
Consumes exactly one  $A$  during computation
- ▶ Exponential type:  $!A$   
Makes elements of  $A$  duplicable and erasable
- ▶ General programs:  
Type  $!A \multimap B$

# Linear Logic: between syntax and semantics

## Syntax

- ▶ Linear programs  $A \multimap B$   
Consumes exactly one  $A$  during computation
- ▶ Exponential type:  $!A$   
Makes elements of  $A$  duplicable and erasable
- ▶ General programs:  
Type  $!A \multimap B$

## Semantics

- ▶ Morphism in  $\mathcal{L}(A, B)$   
 $\mathcal{L}$  is a linear category
- ▶ Comonad  $!$  on  $\mathcal{L}$   
 $!$  is a resource comonad

# Linear Logic: between syntax and semantics

## Syntax

- ▶ Linear programs  $A \multimap B$   
Consumes exactly one  $A$  during computation
- ▶ Exponential type:  $!A$   
Makes elements of  $A$  duplicable and erasable
- ▶ General programs:  
Type  $!A \multimap B$

## Semantics

- ▶ Morphism in  $\mathcal{L}(A, B)$   
 $\mathcal{L}$  is a linear category
- ▶ Comonad  $!$  on  $\mathcal{L}$   
 $!$  is a resource comonad
- ▶ Kleisli category  $\mathcal{L}_!$   
 $\mathcal{L}_!(A, B) = \mathcal{L}(!A, B)$

# Linear Logic: between syntax and semantics

## Syntax

- ▶ Linear programs  $A \multimap B$   
Consumes exactly one  $A$  during computation
- ▶ Exponential type:  $!A$   
Makes elements of  $A$  duplicable and erasable
- ▶ General programs:  
Type  $!A \multimap B$

## Semantics

- ▶ Morphism in  $\mathcal{L}(A, B)$   
 $\mathcal{L}$  is a linear category
- ▶ Comonad  $!$  on  $\mathcal{L}$   
 $!$  is a resource comonad
- ▶ Kleisli category  $\mathcal{L}_!$   
 $\mathcal{L}_!(A, B) = \mathcal{L}(!A, B)$

The Kleisli category  $\mathcal{L}_!$  of the resource comonad  $!_-$  is a cartesian closed category.

In many models of LL: **finiteness spaces**, **Köthe spaces**

- ▶ morphisms in  $\mathcal{L}(X, Y)$ : linear maps between vector spaces
- ▶ morphisms in  $\mathcal{L}_!(X, Y) = \mathcal{L}(!X, Y)$ : analytic maps

In many models of LL: **finiteness spaces**, **Köthe spaces**

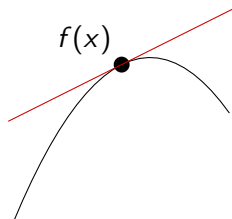
- ▶ morphisms in  $\mathcal{L}(X, Y)$ : linear maps between vector spaces
- ▶ morphisms in  $\mathcal{L}_!(X, Y) = \mathcal{L}(!X, Y)$ : analytic maps

# Differential $\lambda$ -calculus and categorical differentiation

# Differential calculus

A function  $f : \mathbb{R} \rightarrow \mathbb{R}$  is derivable at  $x \in \mathbb{R}$  if

$$f(x + u) = f(x) + f'(x) \cdot u + O(u^2)$$





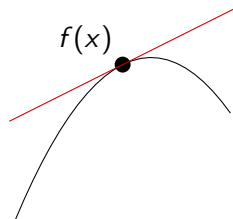
# Differential calculus

A function  $f : \mathbb{R} \rightarrow \mathbb{R}$  is derivable at  $x \in \mathbb{R}$  if

$$f(x + u) = f(x) + f'(x) \cdot u + O(u^2)$$

It is two time derivable at  $x$  if

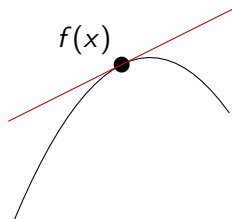
$$f(x + u) = f(x) + f'(x) \cdot u + \frac{1}{2}f^{(2)}(x) \cdot u^2 + O(u^3)$$



# Differential calculus

A function  $f : \mathbb{R} \rightarrow \mathbb{R}$  is derivable at  $x \in \mathbb{R}$  if

$$f(x + u) = f(x) + f'(x) \cdot u + O(u^2)$$



It is two time derivable at  $x$  if

$$f(x + u) = f(x) + f'(x) \cdot u + \frac{1}{2}f^{(2)}(x) \cdot u^2 + O(u^3)$$

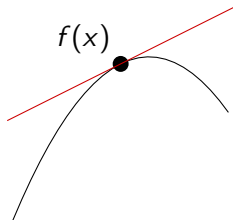
Degree  $n$  Taylor expansion.

$$f(x + u) = \sum_{k=0}^n \frac{1}{k!} f^{(k)}(x) \cdot u^k + O(u^{n+1})$$

# Differential calculus

A function  $f : E \rightarrow F$  is derivable at  $x \in E$  if

$$f(x + u) = f(x) + f'(x) \cdot u + O(\|u\|^2)$$



It is two time derivable at  $x$  if

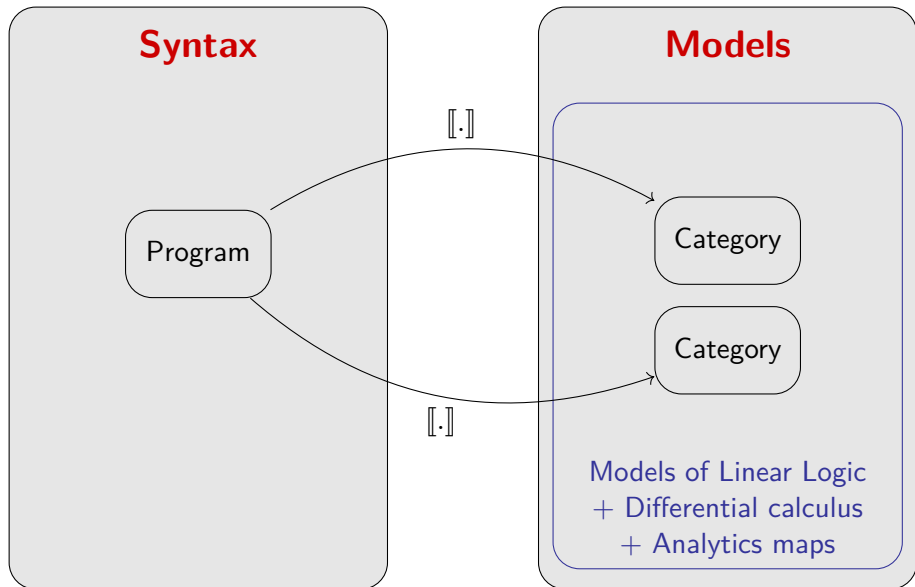
$$f(x + u) = f(x) + f'(x) \cdot u + \frac{1}{2} f^{(2)}(x) \cdot (u, u) + O(\|u\|^3)$$

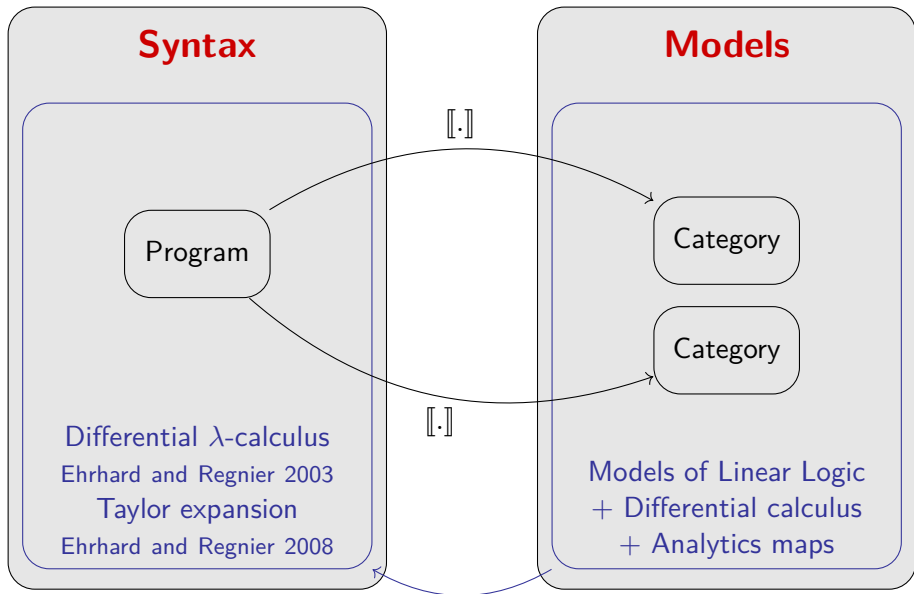
Degree  $n$  Taylor expansion.

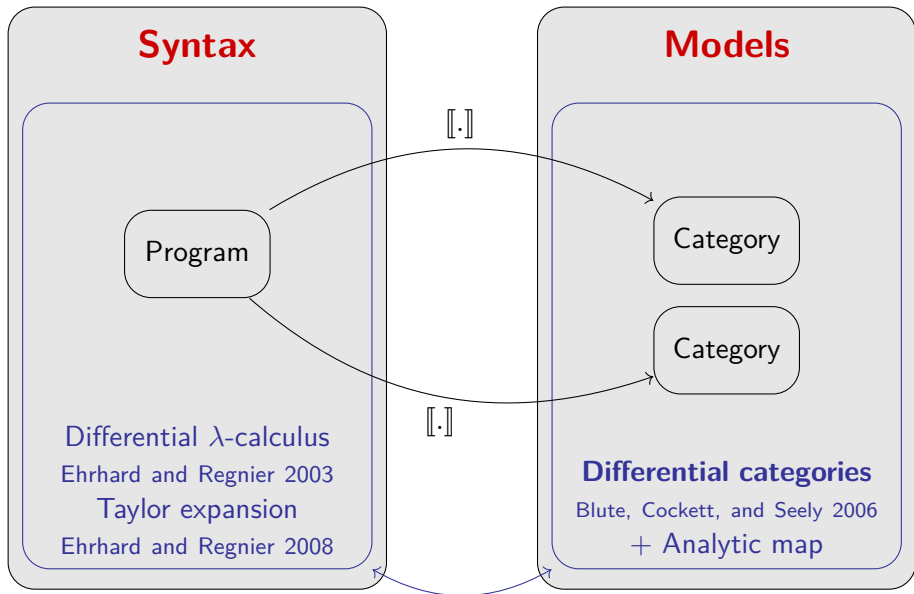
$$f(x + u) = \sum_{k=0}^n \frac{1}{k!} f^{(k)}(x) \cdot \underbrace{(u, \dots, u)}_{k \text{ times}} + O(\|u\|^{n+1})$$

# Analytic map

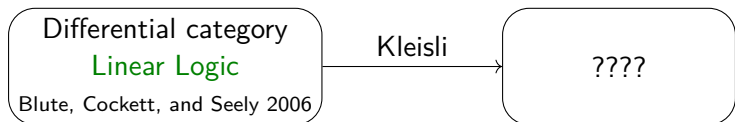
$$f(x + u) = \sum_{k=0}^{\infty} \frac{1}{k!} f^{(k)}(x) \cdot \underbrace{(u, \dots, u)}_{k \text{ times}}$$







# The birth of categorical differentiation





# The birth of categorical differentiation



# The birth of categorical differentiation

Cartesian differential  
category

Blute, Cockett, and Seely 2009

# The birth of categorical differentiation

## Cartesian differential category

Blute, Cockett, and Seely 2009

- ▶ When closed: models of the differential  $\lambda$ -calculus (Bucciarelli, Ehrhard, and Manzonetto 2010) and the syntactical Taylor expansion (Manzonetto12)
- ▶ Related to the semantics of automated differentiation
- ▶ Algebraic theory of differentiation in maths

# The problem of sums

# Sums, sums everywhere...

“Leibniz” rule: if  $f : E \times F \rightarrow G$

$$f'(x, y) \cdot (u, v) = \frac{\partial f}{\partial x}(x, y) \cdot u + \frac{\partial f}{\partial y}(x, y) \cdot v$$

# Sums, sums everywhere...

“Leibniz” rule: if  $f : E \times F \rightarrow G$

$$f'(x, y) \cdot (u, v) = \frac{\partial f}{\partial x}(x, y) \cdot u + \frac{\partial f}{\partial y}(x, y) \cdot v$$

- Semantics: Differential Categories and Cartesian Differential Categories features sums between morphisms;

# Sums, sums everywhere...

“Leibniz” rule: if  $f : E \times F \rightarrow G$

$$f'(x, y) \cdot (u, v) = \frac{\partial f}{\partial x}(x, y) \cdot u + \frac{\partial f}{\partial y}(x, y) \cdot v$$

- ▶ Semantics: Differential Categories and Cartesian Differential Categories features sums between morphisms;
- ▶ The syntax allows **non-determinism**.

# Sums, sums everywhere...

“Leibniz” rule: if  $f : E \times F \rightarrow G$

$$f'(x, y) \cdot (u, v) = \frac{\partial f}{\partial x}(x, y) \cdot u + \frac{\partial f}{\partial y}(x, y) \cdot v$$

- ▶ Semantics: Differential Categories and Cartesian Differential Categories features sums between morphisms;
- ▶ The syntax allows **non-determinism**.

If we want Taylor expansion

...We allow **countable** sums and non-determinism.



# The problem

**The problem in syntax:** Choose, don't endure.

# The problem

**The problem in syntax:** Choose, don't endure.

**The problem in semantics:** Many models of Linear Logic are excluded

- ▶ Some lack finite sums: coherence spaces, probabilistic coherence spaces

# The problem

**The problem in syntax:** Choose, don't endure.

**The problem in semantics:** Many models of Linear Logic are excluded

- ▶ Some lack finite sums: coherence spaces, probabilistic coherence spaces
- ▶ Some lack infinite sums: Köthe spaces, finiteness spaces

# The problem

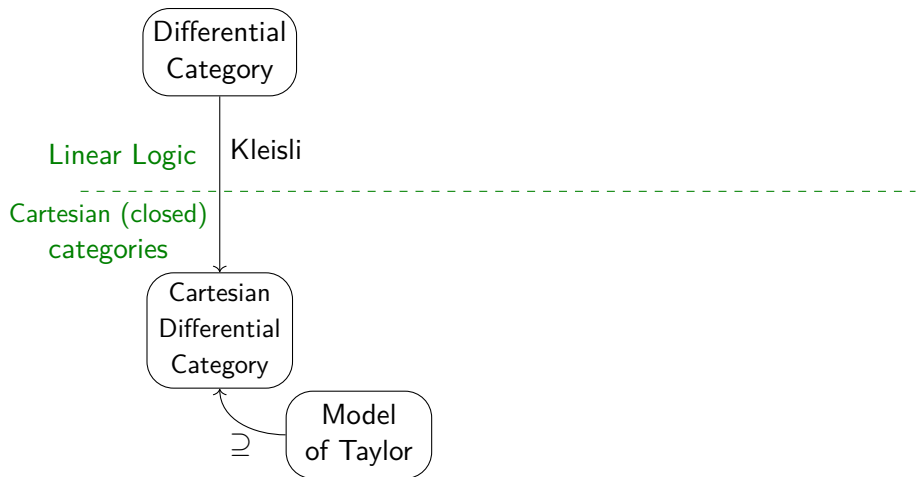
**The problem in syntax:** Choose, don't endure.

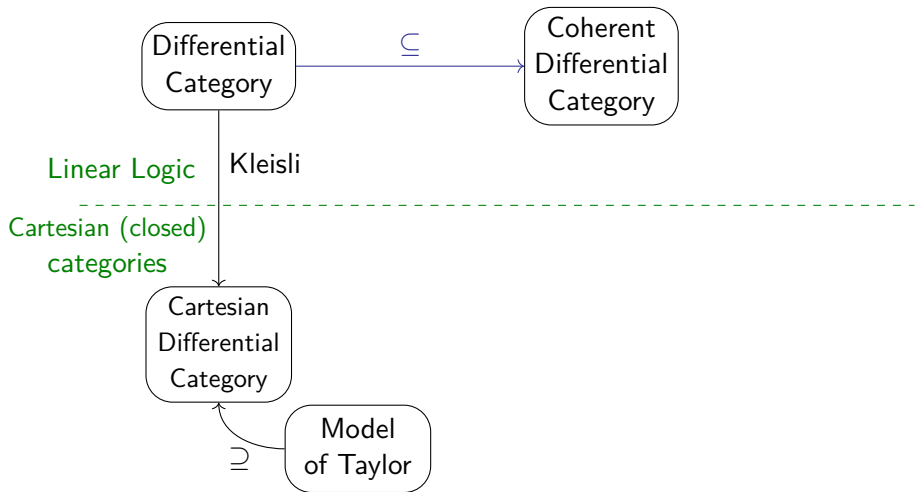
**The problem in semantics:** Many models of Linear Logic are excluded

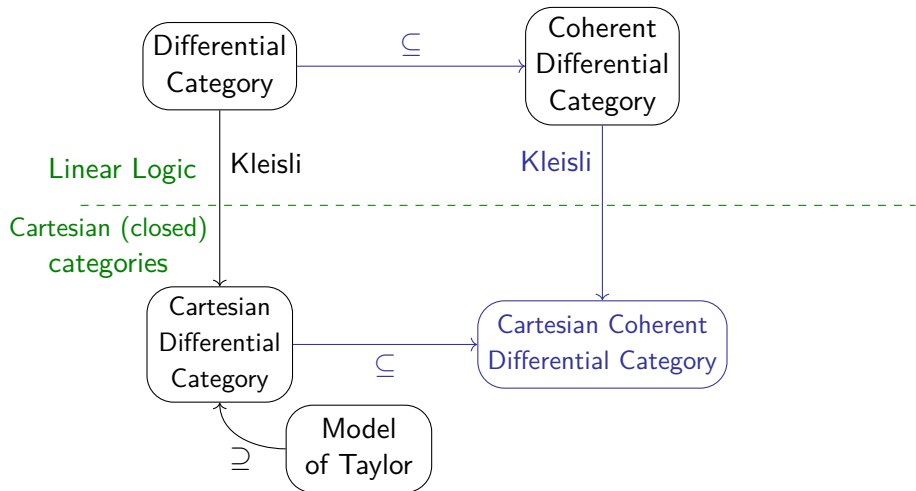
- ▶ Some lack finite sums: coherence spaces, probabilistic coherence spaces
- ▶ Some lack infinite sums: Köthe spaces, finiteness spaces

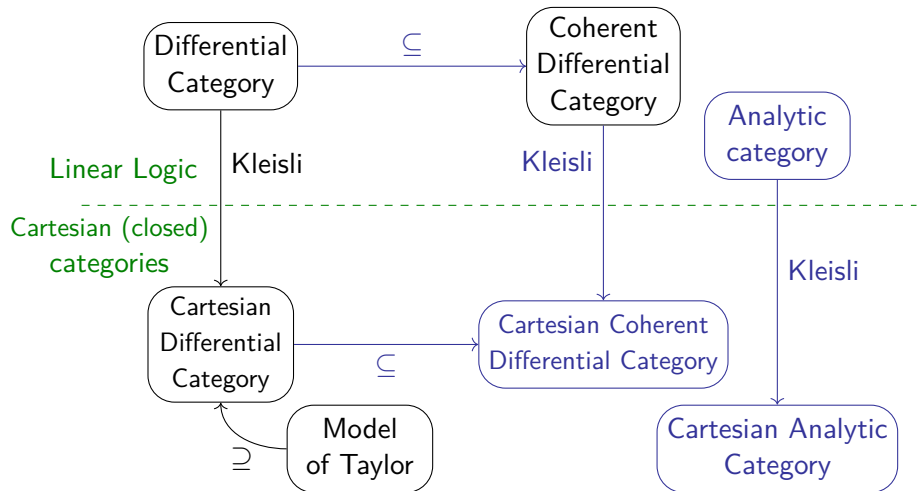
## Objective of the thesis

We want a theory of differential linear logic and Taylor expansion with partial sums.

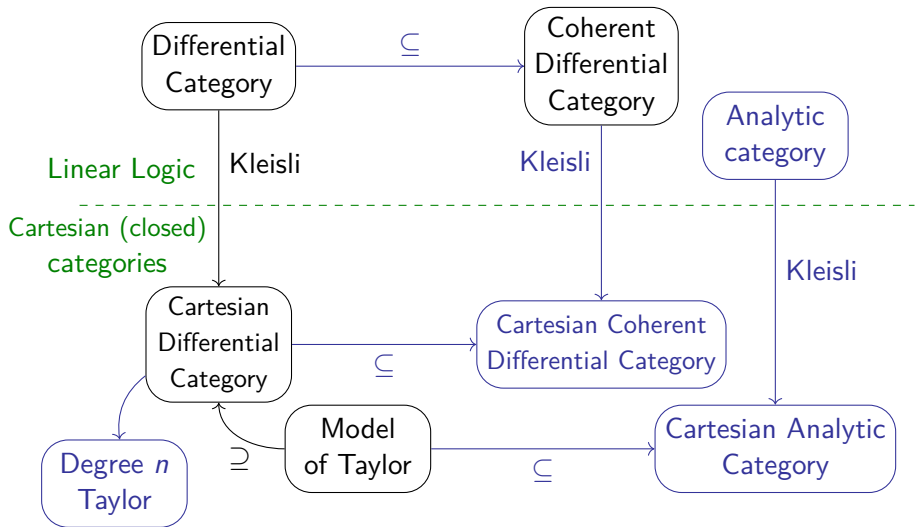


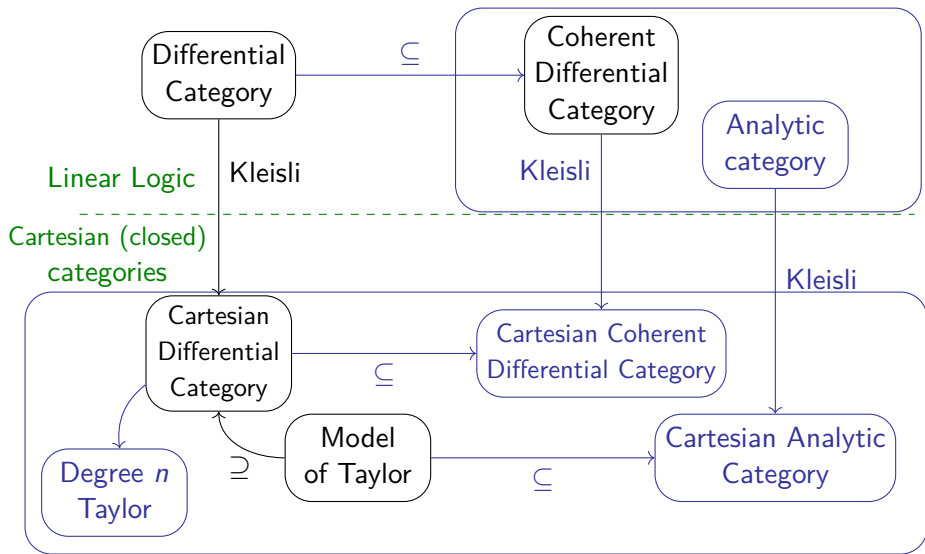












# Compositional Taylor expansion in additive and non-additive models of Linear Logic

We start with  
the derivative

# Cartesian differential category

## Cartesian differential category

Category + sum between morphisms + **derivative operator**

$$\frac{f : X \rightarrow Y}{Df : X \times X \rightarrow Y}$$

+ equations on D (properties of derivatives).

## Example: the category **Smooth**

Morphisms are smooth functions  $f, g : X \rightarrow Y$ .

$$Df : (x, u) \mapsto f'(x) \cdot u$$

## Fixing the chain rule

D must satisfy the chain rule.

$$(g \circ f)'(x) = g'(f(x)) \cdot f'(x)$$



## Fixing the chain rule

D must satisfy the chain rule.

$$(g \circ f)'(x) = g'(f(x)) \cdot f'(x)$$



### Tangent bundle

$$\frac{f : X \rightarrow Y}{\mathsf{T}f : X \times X \rightarrow Y \times Y}$$

$$\mathsf{T}f(x, u) = (f(x), f'(x) \cdot u)$$

## Fixing the chain rule

D must satisfy the chain rule.

$$(g \circ f)'(x) = g'(f(x)) \cdot f'(x)$$



Tangent bundle

$$\frac{f : X \rightarrow Y}{\mathsf{T}f : X \times X \rightarrow Y \times Y}$$

$$\mathsf{T}f(x, u) = (f(x), f'(x) \cdot u)$$

$$\boxed{\mathsf{T}(g \circ f) = \mathsf{T}g \circ \mathsf{T}f}$$





## Fixing the chain rule

D must satisfy the chain rule.

$$(g \circ f)'(x) = g'(f(x)) \cdot f'(x)$$



Tangent bundle

$$\frac{f : X \rightarrow Y}{Tf : TX \rightarrow TY} \quad Tf(x, u) = (f(x), f'(x) \cdot u)$$

$$\boxed{T(g \circ f) = Tg \circ Tf}$$



## Fixing the chain rule

D must satisfy the chain rule.

$$(g \circ f)'(x) = g'(f(x)) \cdot f'(x)$$



### Tangent bundle

$$\frac{f : X \rightarrow Y}{\mathsf{T}f : \mathsf{TX} \rightarrow \mathsf{TY}} \quad \mathsf{T}f(x, u) = (f(x), f'(x) \cdot u)$$

$$\boxed{\mathsf{T}(g \circ f) = \mathsf{T}g \circ \mathsf{T}f}$$



### Dual number intuition

$$(x, u) \in \mathsf{TX} \iff \text{polynomial } x + u\varepsilon \text{ with } \varepsilon^2 = 0$$

## Fixing the chain rule

D must satisfy the chain rule.

$$(g \circ f)'(x) = g'(f(x)) \cdot f'(x)$$



Tangent bundle

$$\frac{f : X \rightarrow Y}{\mathsf{T}f : \mathsf{TX} \rightarrow \mathsf{TY}} \quad \mathsf{T}f(x, u) = (f(x), f'(x) \cdot u)$$

$$\boxed{\mathsf{T}(g \circ f) = \mathsf{T}g \circ \mathsf{T}f}$$



Dual number intuition

$$(x, u) \in \mathsf{TX} \iff \text{polynomial } x + u\varepsilon \text{ with } \varepsilon^2 = 0$$

$$f(x + u\varepsilon) = f(x) + \varepsilon f'(x) \cdot u + O(\varepsilon^2)$$

## Fixing the chain rule

D must satisfy the chain rule.

$$(g \circ f)'(x) = g'(f(x)) \cdot f'(x)$$



### Tangent bundle

$$\frac{f : X \rightarrow Y}{\mathsf{T}f : \mathsf{TX} \rightarrow \mathsf{TY}} \quad \mathsf{T}f(x, u) = (f(x), f'(x) \cdot u)$$

$$\boxed{\mathsf{T}(g \circ f) = \mathsf{T}g \circ \mathsf{T}f}$$



### Dual number intuition

$$(x, u) \in \mathsf{TX} \iff \text{polynomial } x + u\varepsilon \text{ with } \varepsilon^2 = 0$$

$$f(x + u\varepsilon) = f(x) + \varepsilon f'(x) \cdot u + \cancel{O(\varepsilon^2)}$$

# Fixing the chain rule

D must satisfy the chain rule.

$$(g \circ f)'(x) = g'(f(x)) \cdot f'(x)$$



## Tangent bundle

$$\frac{f : X \rightarrow Y}{\mathsf{T}f : \mathsf{TX} \rightarrow \mathsf{TY}} \quad \mathsf{T}f(x, u) = (f(x), f'(x) \cdot u)$$

$$\boxed{\mathsf{T}(g \circ f) = \mathsf{T}g \circ \mathsf{T}f}$$



## Dual number intuition

$$(x, u) \in \mathsf{TX} \iff \text{polynomial } x + u\varepsilon \text{ with } \varepsilon^2 = 0$$

$$f(x + u\varepsilon) = f(x) + \varepsilon f'(x) \cdot u + \cancel{O(\varepsilon^2)} \quad \text{push forward}$$

$Tf(x, u) = (f(x), f'(x) \cdot u)$ : degree 1 Taylor expansion

Let us do the same  
for Taylor expansion

# A degree 2 Taylor expansion

## Tangent bundle

$$TX = X \times X \qquad \frac{f : X \rightarrow Y}{Tf : TX \rightarrow TY}$$

## Dual number intuition

$$(x, u) \in TX \iff \text{polynomial } x + u\varepsilon \text{ with } \varepsilon^2 = 0$$

$Tf(x, u) :$  **push forward** of polynomials

# A degree 2 Taylor expansion

## Degree 2 Taylor bundle

$$T_2X = X \times X \times X \qquad \frac{f : X \rightarrow Y}{T_2f : T_2X \rightarrow T_2Y}$$

## Higher order dual numbers intuition

$$(x, u, v) \in T_2X \iff \text{polynomial } x + u\varepsilon + v\varepsilon^2 \text{ with } \varepsilon^3 = 0$$

$T_2f(x, u, v)$ : **push forward** of polynomial.



# A degree 2 Taylor expansion

## Degree 2 Taylor bundle

$$T_2X = X \times X \times X \qquad \frac{f : X \rightarrow Y}{T_2f : T_2X \rightarrow T_2Y}$$

## Higher order dual numbers intuition

$$(x, u, v) \in T_2X \iff \text{polynomial } x + u\varepsilon + v\varepsilon^2 \text{ with } \varepsilon^3 = 0$$

$T_2f(x, u, v)$ : **push forward** of polynomial.

$$\begin{aligned} & f(x + u\varepsilon + v\varepsilon^2) \\ &= f(x) + f'(x) \cdot (u\varepsilon + v\varepsilon^2) + \frac{1}{2}f^{(2)}(x) \cdot (u\varepsilon + v\varepsilon^2, u\varepsilon + v\varepsilon^2) + \cancel{O(\varepsilon^3)} \end{aligned}$$

# A degree 2 Taylor expansion

## Degree 2 Taylor bundle

$$T_2X = X \times X \times X \qquad \frac{f : X \rightarrow Y}{T_2f : T_2X \rightarrow T_2Y}$$

## Higher order dual numbers intuition

$$(x, u, v) \in T_2X \iff \text{polynomial } x + u\varepsilon + v\varepsilon^2 \text{ with } \varepsilon^3 = 0$$

$T_2f(x, u, v)$ : **push forward** of polynomial.

$$\begin{aligned} & f(x + u\varepsilon + v\varepsilon^2) \\ &= f(x) + f'(x) \cdot (u\varepsilon + v\varepsilon^2) + \frac{1}{2}f^{(2)}(x) \cdot (u\varepsilon + v\varepsilon^2, u\varepsilon + v\varepsilon^2) + \cancel{O(\varepsilon^3)} \\ &= f(x) + \varepsilon f'(x) \cdot u + \varepsilon^2 \left( \frac{1}{2}f^{(2)}(x) \cdot (u, u) + f'(x) \cdot v \right) + \cancel{\varepsilon^3(\dots)} \end{aligned}$$

# A degree 2 Taylor expansion

## Degree 2 Taylor bundle

$$T_2X = X \times X \times X \qquad \frac{f : X \rightarrow Y}{T_2f : T_2X \rightarrow T_2Y}$$

## Higher order dual numbers intuition

$$(x, u, v) \in T_2X \iff \text{polynomial } x + u\varepsilon + v\varepsilon^2 \text{ with } \varepsilon^3 = 0$$

$$T_2f(x, u, v) = \left( f(x), f'(x) \cdot u, \frac{1}{2}f^{(2)}(x) \cdot (u, u) + f'(x) \cdot v \right)$$

$$f(x + u\varepsilon + v\varepsilon^2)$$

$$= f(x) + f'(x) \cdot (u\varepsilon + v\varepsilon^2) + \frac{1}{2}f^{(2)}(x) \cdot (u\varepsilon + v\varepsilon^2, u\varepsilon + v\varepsilon^2) + \cancel{O(\varepsilon^3)}$$

$$= f(x) + \varepsilon f'(x) \cdot u + \varepsilon^2 \left( \frac{1}{2}f^{(2)}(x) \cdot (u, u) + f'(x) \cdot v \right) + \cancel{\varepsilon^3(\dots)}$$

# A compositional Taylor expansion

## Degree $n$ Taylor bundle

$$T_n X = X \times X^n \qquad \frac{f : X \rightarrow Y}{T_n f : T_n X \rightarrow T_n Y}$$

## Higher order dual numbers intuition

$$(x, u_1, \dots, u_n) \in T_n X \iff \text{polynomial } x + \sum_{i=1}^n u_i \varepsilon^i \text{ with } \varepsilon^{n+1} = 0$$

$T_n f(x, u_1, \dots, u_n)$ : **push forward** of polynomial.

# A compositional Taylor expansion

## Degree $n$ Taylor bundle

$$T_n X = X \times X^n \qquad \frac{f : X \rightarrow Y}{T_n f : T_n X \rightarrow T_n Y}$$

## Higher order dual numbers intuition

$$(x, u_1, \dots, u_n) \in T_n X \iff \text{polynomial } x + \sum_{i=1}^n u_i \varepsilon^i \text{ with } \varepsilon^{n+1} = 0$$

$T_n f(x, u_1, \dots, u_n)$ : **push forward** of polynomial.

$$T_n f(x, u_1, \dots, u_n) = \left( \sum_{k=1}^j \sum_{\substack{i_1, \dots, i_k \text{ s.t.} \\ i_1 + \dots + i_k = j}} \frac{1}{k!} f^{(k)}(x) \cdot (u_{i_1}, \dots, u_{i_k}) \right)_{j=0}^n$$

# A compositional Taylor expansion

## Degree $n$ Taylor bundle

$$T_n X = X \times X^n$$

$$\frac{f : X \rightarrow Y}{T_n f : T_n X \rightarrow T_n Y}$$

## Higher order dual numbers intuition

$$(x, u_1, \dots, u_n) \in T_n X \iff \text{polynomial } x + \sum_{i=1}^n u_i \varepsilon^i \text{ with } \varepsilon^{n+1} = 0$$

$T_n f(x, u_1, \dots, u_n)$ : **push forward** of polynomial.

$$T_n f(x, \textcolor{red}{u}, \textcolor{red}{0}, \dots, \textcolor{red}{0}) = \left( \frac{1}{j!} f^{(j)}(x) \cdot (\textcolor{red}{u}, \dots, \textcolor{red}{u}) \right)_{j=0}^n$$

# A compositional Taylor expansion

Faà di Bruno formula: **non-compositional** (and scary)



~~$$(g \circ f)^{(n)}(x) = \sum_{\{l_1, \dots, l_k\} \text{ partition of } [1, n]} g^{(k)}(f(x), f^{(l_1)}(x), \dots, f^{(l_k)}(x))$$~~

with  $f^{(\{i_1, \dots, i_l\})}(x) : (u_1, \dots, u_n) \mapsto f^{(l)}(x)(u_{i_1}, \dots, u_{i_l})$

Theorem 1. (Chapter 6)

$$T_n(g \circ f) = T_n g \circ T_n f$$



# Categorical structure of $T_n$

$T_n$  exists in all **cartesian differential categories**

Theorem 2. (Chapter 5)

$T_n$  is a functor



Chain rule on D



# Categorical structure of $T_n$

$T_n$  exists in all **cartesian differential categories**

## Theorem 2. (Chapter 5)

$T_n$  is a functor  $\iff$  Chain rule on D

$T_n$  is a monad

$z : \text{Id} \Rightarrow T_n, \theta : T_n T_n \Rightarrow T_n$

Distributive law  $c : T_n T_n \Rightarrow T_n T_n \iff$  On axioms on D

Natural transformation

$l : T_n \Rightarrow T_n T_n$

# Categorical structure of $T_n$

$T_n$  exists in all **cartesian differential categories**

Theorem 2. (Chapter 5)

$T_n$  is a functor  $\iff$  Chain rule on D

$T_n$  is a monad

$z : \text{Id} \Rightarrow T_n, \theta : T_n T_n \Rightarrow T_n$

Distributive law  $c : T_n T_n \Rightarrow T_n T_n \iff$  On axioms on D

Natural transformation

$! : T_n \Rightarrow T_n T_n$

$$\pi_n \circ \theta = \sum_{i+j=n} \pi_i \circ \pi_j$$

# Proof by... computation ?

Remember

$$T_n f(x, u_1, \dots, u_n) = \left( \sum_{k=1}^j \sum_{\substack{i_1, \dots, i_k \text{ s.t.} \\ i_1 + \dots + i_k = j}} \frac{1}{k!} f^{(k)}(x) \cdot (u_{i_1}, \dots, u_{i_k}) \right)_{j=0}^n$$

Explicit computation ? ☹️

Category theory proof 😊

Also works for  $n = \infty$

$$T_{\omega}X = X \times X \times \dots$$

Also works for  $n = \infty$

$$T_\omega X = X \times X \times \dots$$

One last ingredient

Morphisms are analytic

$$f(x + u) = \sum_{k \in \mathbb{N}} f^{(k)}(x) \cdot (u, \dots, u)$$

Also works for  $n = \infty$

$$T_{\omega}X = X \times X \times \dots$$

## One last ingredient

Morphisms are analytic

$$f(x + u) = \sum_{k \in \mathbb{N}} f^{(k)}(x) \cdot (u, \dots, u)$$

If and only if

$$f \circ \sigma = \sigma \circ T_{\omega}f$$

where  $\sigma(x_0, x_1, x_2, \dots) = \sum_{i=0}^{\infty} x_i$

Also works for  $n = \infty$

$$T_{\omega}X = X \times X \times \dots$$

One last ingredient

Morphisms are analytic

$$f(x + u) = \sum_{k \in \mathbb{N}} f^{(k)}(x) \cdot (u, \dots, u)$$

If and only if

$$f \circ \sigma = \sigma \circ T_{\omega}f$$

where  $\sigma(x_0, x_1, x_2, \dots) = \sum_{i=0}^{\infty} x_i$

$T_{\omega}$  is a bimonad

# Compositional Taylor expansion in additive and non-additive models of Linear Logic



# Partial sum: change the action on objects

Total sums

$$T_{\omega}X = X \times X \times \dots$$

# Partial sum: change the action on objects

## Total sums

$$T_{\omega}X = X \times X \times \dots$$

## Partial sums

Partial sum on morphisms: **Partial Commutative Monoid (PCM)**

$$T_{\omega}X = \left\{ (x_i)_{i \in \mathbb{N}} \mid \sum_{i \in \mathbb{N}} x_i \text{ is defined} \right\}$$

# Partial sum: change the action on objects

## Total sums

$$T_{\omega}X = X \times X \times \dots$$

## Partial sums

Partial sum on morphisms: **Partial Commutative Monoid (PCM)**

$$T_{\omega}X = \left\{ (x_i)_{i \in \mathbb{N}} \mid \sum_{i \in \mathbb{N}} x_i \text{ is defined} \right\}$$

Formally:  $T_{\omega}X$  is a **summability structure**

- ▶ Projections  $\pi_i : T_{\omega}X \rightarrow X$  jointly monic
- ▶ Sum operation  $\sigma : T_{\omega}X \rightarrow X$
- ▶  $(f_i)_{i \in \mathbb{N}}$  is summable if and only if there is  $f$  such that  $\pi_i \circ f = f_i$

- ▶ Total sums:  $T_\omega X = X \times X \times \dots$
- ▶ Partial sums:  $T_\omega X$  is a summability structure

Same categorical structure:  $T_\omega$  is a bimonad.

$$\pi_n \circ \theta = \sum_{i+j=n} \pi_i \circ \pi_j$$

# Compositional Taylor expansion in additive and non-additive models of Linear Logic

Let  $\mathcal{L}$  be a model of LL.

- ▶ Partial sums on morphism: **Partial Commutative Monoid (PCM)**
- ▶ Summability structure  $SX$

$$SX = \{ (x_i)_{i \in \mathbb{N}} \mid \sum_{i=0}^{\infty} x_i \text{ defined} \}$$

- ▶ Bimonad structure on  $S$ .

Let  $\mathcal{L}$  be a model of LL.

- ▶ Partial sums on morphism: **Partial Commutative Monoid (PCM)**
- ▶ Summability structure  $SX$

$$SX = \{(x_i)_{i \in \mathbb{N}} \mid \sum_{i=0}^{\infty} x_i \text{ defined} \}$$

- ▶ Bimonad structure on  $S$ .

No derivative yet

$$Sf : (x_i)_{i \in \mathbb{N}} \mapsto (f(x_i))_{i \in \mathbb{N}}$$

No derivative yet

$$Sf : (x_i)_{i \in \mathbb{N}} \mapsto (f(x_i))_{i \in \mathbb{N}}$$

Taylor expansion is in the Kleisli category  $\mathcal{L}_!$

Taylor expansion: bimonad  $T$  in  $\mathcal{L}_!$  that **extends**  $S$

$$\begin{array}{ccc} \mathcal{L} & \xrightarrow{S} & \mathcal{L} \\ \text{Der} \downarrow & & \downarrow \text{Der} \\ \mathcal{L}_! & \xrightarrow{T} & \mathcal{L}_! \end{array}$$

1)



## No derivative yet

$$Sf : (x_i)_{i \in \mathbb{N}} \mapsto (f(x_i))_{i \in \mathbb{N}}$$

Taylor expansion is in the Kleisli category  $\mathcal{L}_!$

Taylor expansion: bimonad  $T$  in  $\mathcal{L}_!$  that **extends**  $S$

$$\begin{array}{ccc} \mathcal{L} & \xrightarrow{S} & \mathcal{L} \\ \text{Der} \downarrow & & \downarrow \text{Der} \\ \mathcal{L}_! & \xrightarrow{T} & \mathcal{L}_! \end{array}$$

- 1) The bimonad structure on  $T$  is the same as the bimonad structure on  $S$

This is the same

- ▶ Bimonad  $T$  that extends  $S$
- ▶ Natural transformation  $\partial : !S \Rightarrow S!$ 
  - ▶ **Distributive law** between comonad  $!_-$  and functor  $S$
  - ▶ Bimonad structure on  $S$ : **morphisms of distributive laws**

This is the same

- ▶ Bimonad  $T$  that extends  $S$
- ▶ Natural transformation  $\partial : !S \Rightarrow S!$ 
  - ▶ **Distributive law** between comonad  $!_-$  and functor  $S$
  - ▶ Bimonad structure on  $S$ : **morphisms of distributive laws**

$$\begin{array}{ccccc} !SS & \xrightarrow{\partial S} & S!S & \xrightarrow{S\partial} & SS! \\ !\theta \downarrow & & & & \downarrow \theta! \\ !S & \xrightarrow{\quad \partial \quad} & S! & & \end{array}$$

This is the same

- ▶ Bimonad  $T$  that extends  $S$
- ▶ Natural transformation  $\partial : !S \Rightarrow S!$ 
  - ▶ **Distributive law** between comonad  $!_-$  and functor  $S$
  - ▶ Bimonad structure on  $S$ : **morphisms of distributive laws**

$$\begin{array}{ccccc}
 !SS & \xrightarrow{\partial S} & S!S & \xrightarrow{S\partial} & SS! \\
 !\theta \downarrow & & & & \downarrow \theta! \\
 !S & \xrightarrow{\quad \partial \quad} & S! & & 
 \end{array}$$

Theorem 3. (Chapter 9)

The Kleisli category  $\mathcal{L}_S$  of the **monad**  $S$  is a model of LL.

# Time to build concrete models!

# The representable theory

Introduced by Ehrhard 2023 for coherent differentiation.

# The representable theory

Introduced by Ehrhard 2023 for coherent differentiation.

$$SX = \mathbb{D} \multimap X \quad \text{with} \quad \mathbb{D} = \bigwedge_{i \in \mathbb{N}} 1$$

# The representable theory

Introduced by Ehrhard 2023 for coherent differentiation.

$$SX = \mathbb{D} \multimap X \quad \text{with} \quad \mathbb{D} = \big\&_{i \in \mathbb{N}} 1$$

What does that means?

$$\text{Injection } \iota_i = (\overbrace{0, 0, \dots, 0}^{i-1}, 1, 0, \dots) \in \mathbb{D}$$



# The representable theory

Introduced by Ehrhard 2023 for coherent differentiation.

$$SX = \mathbb{D} \multimap X \quad \text{with} \quad \mathbb{D} = \bigotimes_{i \in \mathbb{N}} 1$$

What does that means?

$$\text{Injection } \iota_i = (\overbrace{0, 0, \dots, 0}^{i-1}, 1, 0, \dots) \in \mathbb{D}$$

**Epicity assumption:** An element  $s \in \mathbb{D} \multimap X$  is the same as  $(s(\iota_i))_{i \in \mathbb{N}}$

# The representable theory

Introduced by Ehrhard 2023 for coherent differentiation.

$$SX = \mathbb{D} \multimap X \quad \text{with} \quad \mathbb{D} = \bigotimes_{i \in \mathbb{N}} 1$$

What does that means?

$$\text{Injection } \iota_i = (\overbrace{0, 0, \dots, 0}^{i-1}, 1, 0, \dots) \in \mathbb{D}$$

**Epicity assumption:** An element  $s \in \mathbb{D} \multimap X$  is the same as  $(s(\iota_i))_{i \in \mathbb{N}}$

$$\sum_{i \in \mathbb{N}} s(\iota_i) = s(1, 1, 1, \dots)$$

# A recipe for guessing sums

## In coherence spaces

A family of clique is summable if

- ▶ The union is a clique
- ▶ The intersection are pairwise disjoint

# A recipe for guessing sums

## In coherence spaces

A family of clique is summable if

- ▶ The union is a clique
- ▶ The intersection are pairwise disjoint

## In probabilistic coherence spaces

A family of sub probability distributions is summable if the sum is also a sub probability distribution.

# Lots of models !

## Theorem 4. (Chapter 10)

*Every Lafont model of Linear Logic such that  $\mathbb{D} \multimap \_$  is well-defined has a Taylor expansion.*

Proof: distributive laws, adjunctions, **Mates**

# Lots of models !

## Theorem 4. (Chapter 10)

*Every Lafont model of Linear Logic such that  $\mathbb{D} \multimap \_$  is well-defined has a Taylor expansion.*

Proof: distributive laws, adjunctions, **Mates**

- ▶ Relational models
- ▶ Weighted relational models
- ▶ Coherence spaces (Girard, non uniform, weak)
- ▶ Probabilistic coherence spaces
- ▶ **WIP**: Köthe spaces and finiteness spaces?

To conclude

# Conclusion

A **compositional** theory of Taylor expansion

- ▶ With total or partial sums
- ▶ In cartesian (closed) categories or in models of LL



# Conclusion

A **compositional** theory of Taylor expansion

- ▶ With total or partial sums
- ▶ In cartesian (closed) categories or in models of LL

## Future work

Linear Logic flavour:

- ▶ A syntax of coherent Taylor expansion
- ▶ More models **Köthe spaces**, **Finiteness spaces**
- ▶ Alternative bimonads  $\mathbb{D} \multimap \_$

Cartesian (closed) categories flavour:

- ▶ Relate with higher order automated differentiation
- ▶ Axiomatization in tangent categories: **jet bundles**

- 1 Denotational semantics
- 2 Linear Logic
- 3 Compositional Taylor expansion
- 4 ... in Linear Logic
- 5 The end



Girard, Jean-Yves (1987). "Linear Logic". In: Theoretical Computer Science 50, pp. 1–102. DOI: [10.1016/0304-3975\(87\)90045-4](https://doi.org/10.1016/0304-3975(87)90045-4). URL: [https://doi.org/10.1016/0304-3975\(87\)90045-4](https://doi.org/10.1016/0304-3975(87)90045-4).



Blute, R., Robin Cockett, and R. Seely (Dec. 2006). "Differential categories". In: Mathematical Structures in Computer Science 16, pp. 1049–1083. DOI: [10.1017/S0960129506005676](https://doi.org/10.1017/S0960129506005676).



Ehrhard, Thomas and Laurent Regnier (2003). "The differential lambda-calculus". In: Theoretical Computer Science 309.1-3, pp. 1–41.



— (2008). "Uniformity and the Taylor expansion of ordinary lambda-terms". In: Theoretical Computer Science 403.2, pp. 347–372. ISSN: 0304-3975. DOI: <https://doi.org/10.1016/j.tcs.2008.06.001>. URL: <https://www.sciencedirect.com/science/article/pii/S0304397508004064>.



Bucciarelli, Antonio, Thomas Ehrhard, and Giulio Manzonetto (2010). "Categorical Models for Simply Typed Resource Calculi". In: Electronic Notes in Theoretical Computer Science 265. Proceedings of the 26th Conference on the Mathematical Foundations of Programming Semantics (MFPS 2010), pp. 213–230. ISSN: 1571-0661. DOI: <https://doi.org/10.1016/j.entcs.2010.08.013>. URL: <https://www.sciencedirect.com/science/article/pii/S1571066110000927>.



Blute, R., Robin Cockett, and R. Seely (Jan. 2009). "Cartesian differential categories". In: Theory and Applications of Categories 22, pp. 622–672.



Ehrhard, Thomas (2023). "Coherent differentiation". In:  
Mathematical Structures in Computer Science, pp. 1–52. DOI:  
[10.1017/S0960129523000129](https://doi.org/10.1017/S0960129523000129).